DISCOVERING LATENT KNOWLEDGE IN LANGUAGE MODELS WITHOUT SUPERVISION

Collin Burns[*] UC Berkeley
Haotian Ye[*] Peking University
Dan Klein UC Berkeley
Jacob Steinhardt UC Berkeley

arXiv:2212.03827v2 [cs.CL] 2 Mar 2024

ABSTRACT

Existing techniques for training language models can be misaligned with the truth: if we train models with imitation learning, they may reproduce errors that humans make; if we train them to generate text that humans rate highly, they may output errors that human evaluators can't detect. We propose circumventing this issue by directly finding latent knowledge inside the internal activations of a language model in a purely unsupervised way. Specifically, we introduce a method for accurately answering yes-no questions given only unlabeled model activations. It works by finding a direction in activation space that satisfies logical consistency properties, such as that a statement and its negation have opposite truth values. We show that despite using no supervision and no model outputs, our method can recover diverse knowledge represented in large language models: across 6 models and 10 question-answering datasets, it outperforms zero-shot accuracy by 4% on average. We also find that it cuts prompt sensitivity in half and continues to maintain high accuracy even when models are prompted to generate incorrect answers. Our results provide an initial step toward discovering what language models know, distinct from what they say, even when we don't have access to explicit ground truth labels.

1 INTRODUCTION

The increasing deployment of language models in real-world applications opens up exciting possibilities, but it also raises the stakes of AI research and presents new risks (Bommasani et al., 2021; Weidinger et al., 2021; Bender et al., 2021). One of these risks is that language models do not always output text that is true (Evans et al., 2021; Hendrycks et al., 2021; Kenton et al., 2021).

Common training objectives can cause models to learn internal representations related to truth, since truth is a useful feature for many tasks. However, these objectives can also cause language models to output text that is false, at least in some circumstances. For example, if we train a model to imitate human-generated text, it may learn to output common misconceptions (Lin et al., 2022). Or if we train a chat bot to optimize a reward such as engagement, it may learn to generate text that is compelling but false (Roller et al., 2021). If we try to reward model outputs that look true, a model may still learn to output false text if human raters can't evaluate the correctness of that text (Kenton et al., 2021).

In each case, this is an issue that stems from the misalignment between a training objective and the truth. As models are applied to more complex domains, human supervision may become less effective at mitigating this misalignment. Moreover, because this is a problem with the training objective rather than a model's capabilities, it likely won't be solved by scaling up models alone.

We propose a different approach for addressing this misalignment: using models to answer questions in a purely *unsupervised* way. Intuitively, instead of trying to explicitly, externally specify truth, we search for implicit, internal "beliefs" or "knowledge" learned by a model. We approach this problem by leveraging the fact that a model's representation of truth must satisfy logical consistency properties, which are unlikely to be satisfied by many other features.

We implement this idea by introducing Contrast-Consistent Search (CCS), a method that learns a linear projection of the hidden states that is consistent across negations, as illustrated in Figure 1. We find that despite its simplicity and despite not having access to any labels or model outputs,

[*]Equal contribution.

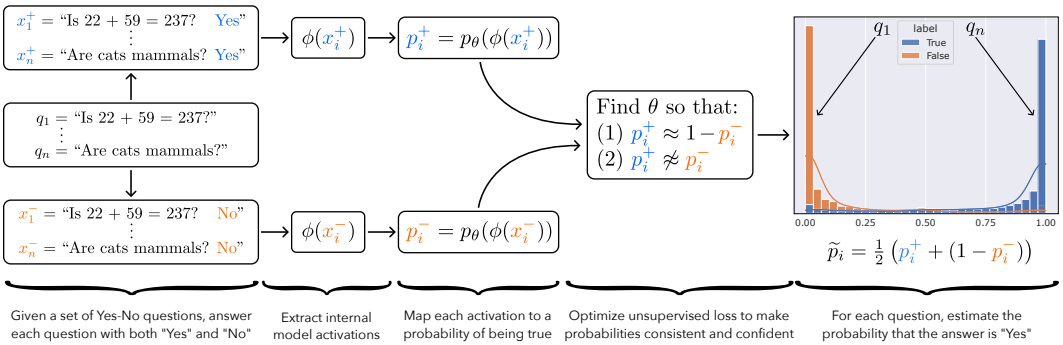

Figure 1: An illustration of our method, Contrast-Consistent Search (CCS). For each yes-no question q_i, we let x_i^+ and x_i^- be the natural language statements where we answer q_i as "Yes" and "No" respectively. Answering the question q_i then amounts to determining which of x_i^+ or x_i^- is true. We compute probabilities p_i^+ and p_i^- that x_i^+ and x_i^- are true respectively using a learned mapping from the hidden states to a number between 0 and 1. We search for a mapping such that that the probabilities are both confident and consistent. On the right, we show a histogram of the "Yes" probabilities, $\tilde{p}_i = 0.5 \cdot (p_i^+ + (1 - p_i^-))$, learned by our method on the unlabeled train split of the COPA dataset (Roemmele et al., 2011) with the UnifiedQA model (Khashabi et al., 2020). Our method uses no labels and no model outputs, but still learns to accurately answers questions.

CCS can accurately recover knowledge from model representations: evaluated across 6 models and 10 question-answering datasets, CCS outperforms the accuracy of strong zero-shot baselines by 4% on average (Section 3.2.1). The resulting classifier is also less sensitive to different prompts than zero-shot, cutting the standard deviation in accuracy in half. Additionally, we try deliberately prompting models to make incorrect outputs, which should intuitively change what models say but which shouldn't affect their latent knowledge. We find that this causes zero-shot accuracy to drop by up to 9.5% (Section 3.2.2) without decreasing the accuracy of CCS.

We systematically analyze CCS to understand the features it discovers. We show that it transfers across unrelated tasks, suggesting that models may have a task-agnostic representation of the truth and that CCS is able to approximately discover it (Section 3.3.1). Moreover, CCS sometimes works best using the hidden states in the *middle* layers of a network and can work even when model outputs aren't very informative, suggesting that it can leverage different features from those used by the outputs (Section 3.3.2). Finally, we show that representations of truth tend to be salient in models: they can often be found without much data, and they can often be found by taking the top principal component of a slightly modified representation space (Section 3.3.3).

Most existing techniques for making models truthful use human supervision to explicitly specify what is correct. However, it is not feasible to provide supervision in some settings. Our work serves as a proof of concept that an external source of ground truth may not actually be necessary: we may instead be able to find a model's latent representation of truth, independent of what a model says, without using any supervision in the first place.

2 Problem Statement and Framework

In this section we describe our problem setup in more detail and introduce Contrast-Consistent Search (CCS), a method for discovering latent knowledge in language models without supervision.

2.1 Problem: Discovering Latent Knowledge

Given a pre-trained neural language model and a set $q_1, \ldots, q_n$ of yes-no questions[1], our goal is to answer each q_i correctly. Here, q_i can be any question with a well-defined answer, including procedural questions like "Is 22+59 = 237?", for which the answer is "No", and factual questions like "Are cats mammals?", for which the answer is "Yes".

[1]Technically, we only require that there are two mutually exclusive answers. For example, we can also use the labels "positive" and "negative" for sentiment classification. Moreover, our setup can easily extend to the case where we want to evaluate the truth of a set of statements instead of answering a set of questions.

Importantly, we want methods that do not rely on the model generating correct outputs and that do not rely on external supervision. Instead, we turn to the model's unlabeled hidden representations. Specifically, let $\phi(x) \in \mathbb{R}^d$ denote some feature representation on a natural langauge input x, such as the hidden states of a Transformer-based language model. Our goal is to answer the questions $q_1, \ldots, q_n$ only given access to $\phi(\cdot)$. In Section 2.2 we introduce a method for this problem that attains high accuracy (Section 3), demonstrating that this task is tractable.

2.2 METHOD: CONTRAST-CONSISTENT SEARCH

To make progress on the goal described above, we exploit the fact that truth has special structure: it satisfies consistency properties that few other features in a language model are likely to satisfy. Our method, Contrast-Consistent Search (CCS), leverages this idea by finding a direction in activation space that is consistent across negations. As we illustrate in Figure 1, CCS works by (1) answering each question q_i as both "Yes" (x_i^+) and "No" (x_i^-), (2) computing the representations $\phi(x_i^+)$ and $\phi(x_i^-)$ of each answer, (3) mapping the answer representations to probabilities p_i^+ and p_i^- of being true, then (4) optimizing that mapping so that the probabilities are both consistent and confident.

Concretely, the input to CCS is a set of Yes-No questions, $q_1, \ldots, q_n$, and access to a pretrained model's representations, $\phi(\cdot)$; the output of CCS is a lightweight probe on top of $\phi(\cdot)$ that can answer new questions. Here, $\phi(\cdot)$ is fixed but should contain useful information about the answers to $q_1, \ldots, q_n$, in the sense that if one *did* (hypothetically) have access to the ground-truth labels for $q_1, \ldots, q_n$, one would be able to train a small supervised probe on $\phi(\cdot)$ that attains high accuracy. Importantly, CCS does not modify the weights of the pretrained model and it does not use labels.

Constructing contrast pairs. An important property that truth satisfies is negation consistency: the answer to a clear-cut question cannot be both "Yes" and "No" at the same time, as these are negations of each other. Probabilistically, for each question q_i, the probability that the answer to q_i is "Yes" should be one minus the probability that the answer to q_i is "No". To use this property, we begin by constructing contrast pairs: for each question q_i, we answer q_i both as "Yes", resulting in the new natural language statement x_i^+, and as "No", resulting in the natural language statement x_i^-. We illustrate this in Figure 1 (left). We will then learn to classify x_i^+ and x_i^- as true or false; if x_i^+ is true, then the answer to q_i should be "Yes", and if x_i^- is true, then the answer to q_i should be "No".

In practice, we convert each task into a question-answering task with two possible labels, then we use task-specific zero-shot prompts to format questions and answers as strings to construct each contrast pair. The opposite labels we use to construct contrast pairs can be "Yes" and "No" for a generic task, or they can be other tasks-specific labels, such as "Positive" and "Negative" in the case of sentiment classification. We describe the exact prompts we use to for each task in Appendix B.

Feature extraction and normalization. Given a contrast pair (x_i^+, x_i^-), CCS first computes the representations $\phi(x_i^+)$ and $\phi(x_i^-)$ using the feature extractor $\phi(\cdot)$. Intuitively, there are two salient differences between $\phi(x_i^+)$ and $\phi(x_i^-)$: (1) x_i^+ ends with "Yes" while x_i^- ends with "No", and (2) one of x_i^+ or x_i^- is true while the other is false. We want to find (2) rather than (1), so we first try to remove the effect of (1) by normalizing $\{\phi(x_i^+)\}$ and $\{\phi(x_i^-)\}$ independently. In particular, we construct normalized representations $\tilde{\phi}(x)$ as follows:

$$\tilde{\phi}(x_i^+) := \phi(x_i^+) - \mu^+ ; \quad \tilde{\phi}(x_i^-) := \phi(x_i^-) - \mu^-,$$

where $\mu^+, \mu^- \in \mathbb{R}^d$ are the means of $\{\phi(x_i^+)\}_{i=1}^n$ and $\{\phi(x_i^-)\}_{i=1}^n$. This normalization ensures that $\{\tilde{\phi}(x_i^+)\}$ and $\{\tilde{\phi}(x_i^-)\}$ no longer form two separate clusters. In practice we also normalize the scale of the features, but this isn't essential for the method to work; see Appendix G.1 for details.

Mapping activations to probabilities. Next, we learn a probe $p_{\theta,b}(\tilde{\phi})$ that maps a (normalized) hidden state $\tilde{\phi}(x)$ to a number between 0 and 1 representing the probability that the statement x is true. We use a linear projection followed by a sigmoid $\sigma(\cdot)$, i.e. $p_{\theta,b}(\tilde{\phi}) = \sigma(\theta^T \tilde{\phi} + b)$, but nonlinear projections can also work. For simplicity, we sometimes omit the θ, b subscript in p.

Training objective. To find features that represent the truth, we leverage the consistency structure of truth. First, we use the fact that a statement and its negation should have probabilities that add up to 1. This motivates the consistency loss:

$$L_{\text{consistency}}(\theta, b; q_i) := \left[p_{\theta,b}(x_i^+) - (1 - p_{\theta,b}(x_i^-))\right]^2$$